hashlock.

Security Audit

Mavryk (DeFi)

September 2025 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

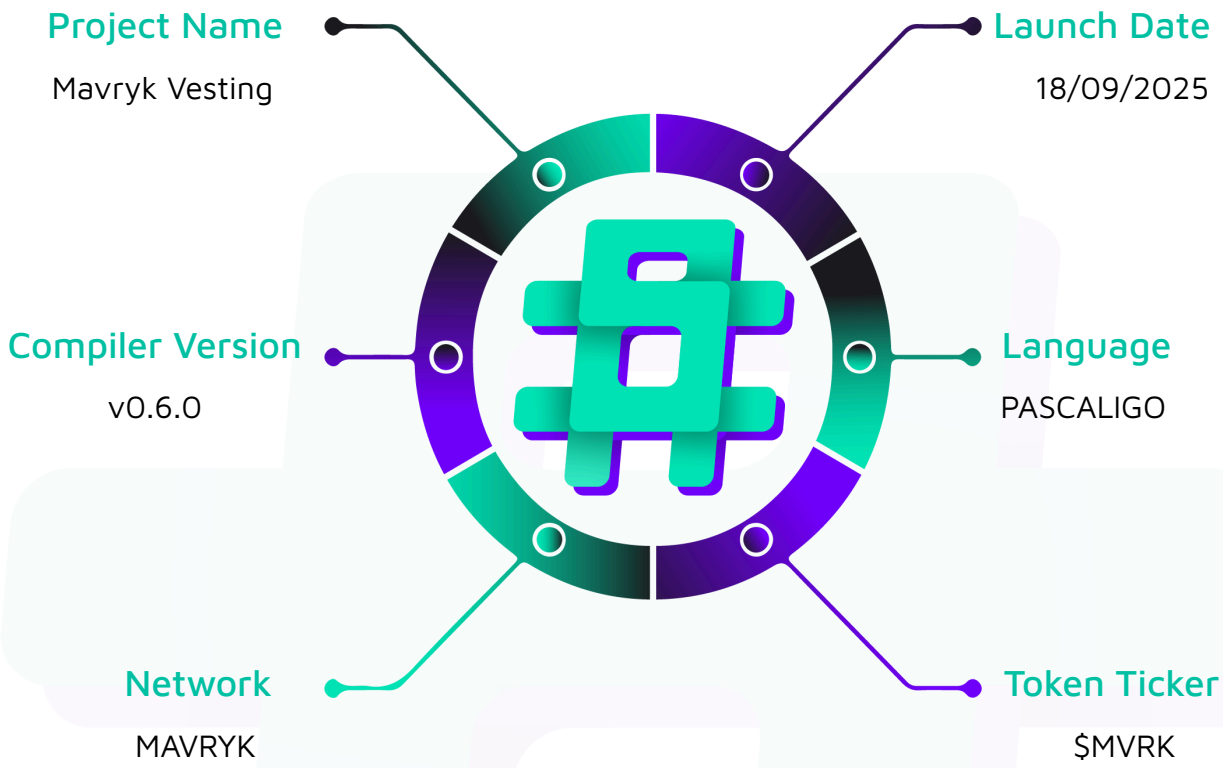
The Mavryk team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

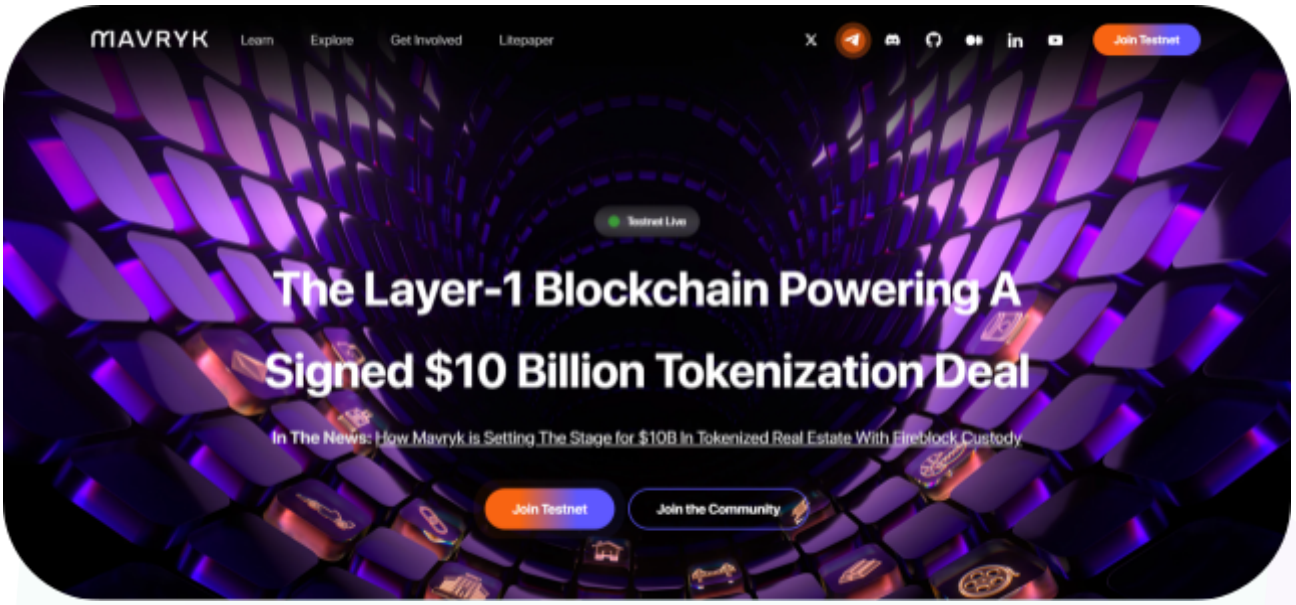
Mavryk is an emerging blockchain platform and ecosystem centered on a purpose-built Layer-1 network (powered by the $MVRK token) that brings together DeFi, real-world asset tokenization, and on-chain governance. Through its tools—such as the Mavryk Wallet browser extension, which facilitates swapping, payments, and DeFi interactions in a Web3-native format—and the Mavkit toolkit (including nodes, protocol implementations, and developer libraries), Mavryk enables seamless asset management, extensibility through self-amending protocols, and participation in staking, governance, and tokenized finance.

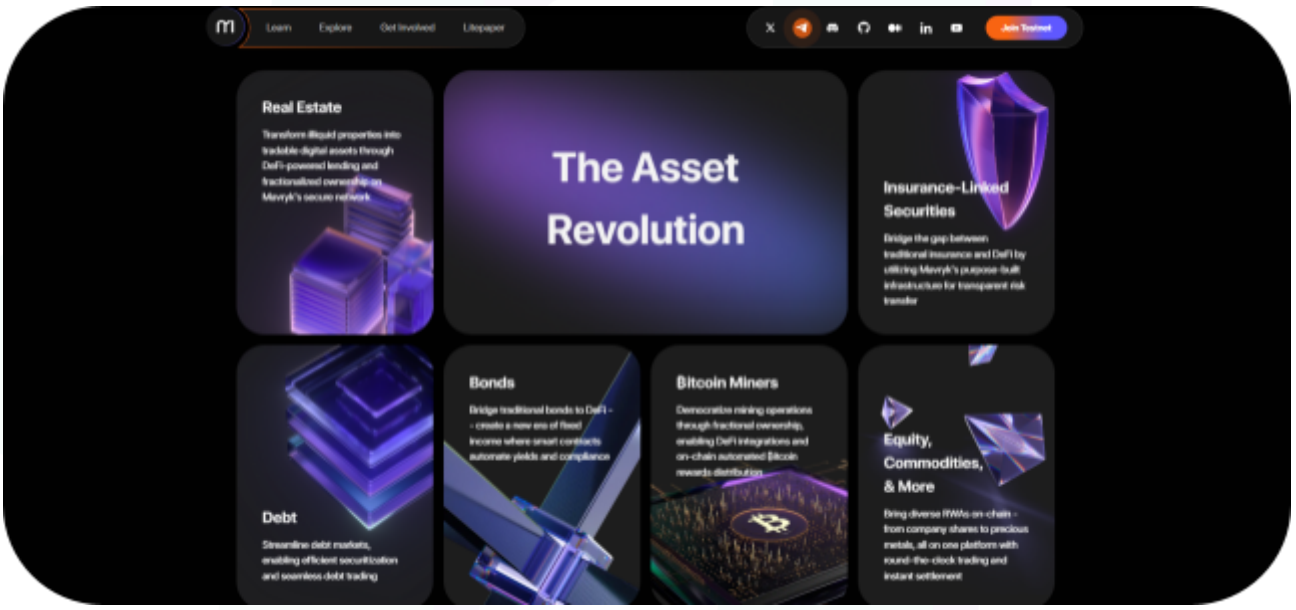
Project Name: Mavryk Vesting
Project Type: DeFi
Compiler Version: 0.6.0
Website: https://mavryk.org/
Logo:

Visualised Context:



Project Visuals:





Audit Scope

We at Hashlock audited the Mavryk code within the Mavryk Vesting project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**Mavryk Vesting Smart Contracts**
Platform	**Mavryk / Pascaligo**
Audit Date	**September, 2025**
Component 1	src/contracts/contracts/main/: - Treasury.ligo - vesting.ligo - vestingFactory.ligo

Component 2	src/contracts/contracts/partials/:
	- contractEntrypoints
	  - treasuryEntrypoints.ligo
	  - vestingEntrypoints.ligo
	  - vestingFactoryEntrypoints.ligo
	- contractHelpers
	  - treasuryHelpers.ligo
	  - vestingFactoryHelpers.ligo
	  - vestingHelpers.ligo
	- contractLambdas
	  - treasuryLambdas.ligo
	  - vestingFactoryLambdas.ligo
	  - vestingLambdas.ligo
	- contractTypes
	  - treasuryTypes.ligo
	  - vestingFactoryTypes.ligo
	  - vestingTypes.ligo
	- contractViews
	  - treasuryViews.ligo
	  - vestingFactoryViews.ligo
	  - vestingViews.ligo
	- Shared
	  - Constants.ligo
	  - multisigHelpers.ligo

	- multisigTypes.ligo - sharedHelpers.ligo - sharedTypes.ligo - transferHelpers.ligo - transferTypes.ligo
Audited GitHub Commit Hash	4ba66dbd0a9840023e0902705a90eca4fff997ba
Fix Review GitHub Commit Hash	325f2b69707e4e82d36a05d12194b3130b1750c1

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

1 High severity vulnerability

1 Low severity vulnerability

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
treasury.ligo - Multisig treasury for MAV and token transfers - Propose and trigger admin actions with thresholds - Update admins, baker, metadata, and config - Build typed transfers for MAV, FA1.2, and FA2	**Contract achieves this functionality.**
vesting.ligo - Hold vesting schedules and balances - Compute claim eligibility and amounts - Execute factory-supplied lambdas on calls - Manage vestee add, remove, lock, and metadata - Default entrypoint handles baker-sent MAV	**Contract achieves this functionality.**
vestingFactory.ligo - Multisig factory to manage vesting contracts - Store and serve lambda bytecode by name - Maintain signatories and contracts, and admin ledgers - Create vestings and map vestees to contracts - Batch actions for claim, set baker, set factory	**Contract achieves this functionality.**
contractEntrypoints - Public entrypoints that load and run lambdas - Route params to matching Lambda actions - Basic sender and amount checks at entry	**Contract achieves this functionality.**
contractHelpers - Permission checks for signatories and admins - Entrypoint discovery and address utilities	**Contract achieves this functionality.**

- Vesting math and state transition helpers	
contractLambdas - Business logic executed from storage bytes - Vesting claim and vestee lifecycle lambdas - Factory creates, adds, claims, toggles, and transfers lambdas - Treasury admin, transfer, baker, metadata lambdas	**Contract achieves this functionality.**
contractTypes - Storage records for factory, vesting, treasury - Lambda action unions and parameter structs - Shared enums, actio, and result types	**Contract achieves this functionality.**
contractViews - Read-only getters for on-chain state - Resolve vestee, vesting, and admin info - Inspect proposals, metadata, and balances	**Contract achieves this functionality.**
shared - Common helpers for lambda lookup and unpack - Multisig proposal pack and unpack utilities - Token transfer builders for MAV, FA1.2, and FA2 - Constants for errors, timings, and names	**Contract achieves this functionality.**

Code Quality

This audit scope involves the smart contracts of the Mavryk Vesting project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices, and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Mavryk Vesting project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] main/vestingFactory.ligo#_verifySenderIsVestingAdmin - Contract-admin tuple mismatch blocks contract specific admins

Description

Factory stores contract-admin keys as `(adminAddress, contractAddress)` in `contractAdminLedger`, but `_verifySenderIsVestingAdmin` builds and checks `(vestingAddress, sender)`. This reversed tuple causes legitimate contract-specific admins to fail authorization while general admins still pass.

Vulnerability Details

The factory stores contract-admin keys as a tuple shaped like `(adminAddress, contractAddress)` via helpers and admin lambdas. The permission gate `_verifySenderIsVestingAdmin` constructs the tuple in the reverse order `(vestingAddress, sender)` when probing `contractAdminLedger`.

The resulting key mismatch means a correctly authorized contract-specific admin is never found during checks. General admins still pass which hides the defect until contract admins attempt to operate vesting actions like `lambdaAddVestee`, `lambdaSetBaker` or `lambdaToggleVesteeLock`.

```
// verify sender is vesting admin

function _verifySenderIsVestingAdmin(const vestingAddress : address; const s : vestingFactoryStorageType) : unit is

block {



    const sender : address = Mavryk.get_sender();

    const userIsGeneralAdmin : bool = case s.generalAdminLedger[sender] of [
```

```
            Some(_v) -> True
        |   None     -> False
    ];

    const vestingAdminTuple : (address * address) = (vestingAddress, sender);
    const userIsContractAdmin : bool = case s.contractAdminLedger[vestingAdminTuple] of [
            Some(_) -> True
        |   None    -> False
    ];

    // pass if user is general admin or contract admin
                if    userIsGeneralAdmin    or    userIsContractAdmin    then    skip    else
failwith("error_NOT_ADMIN");

} with unit
```

Impact

Contract admins cannot perform factory-gated actions. This causes an operational denial of service and may push operators to grant unnecessary general-admin privileges.

Recommendation

We recommend normalizing the tuple order across storage and checks. Build the lookup tuple as `(sender, vestingAddress)` in `_verifySenderIsVestingAdmin` or migrate the ledger and all stores to `(contractAddress, adminAddress)` consistently.

Status

Resolved

Low

[L-01] main/vestingFactory.ligo#setVestingLambda - Live lambda updates allow immediate behavior takeover

Description

Vesting entrypoints resolve lambda bytes from the factory on every call using a name key in `vestingLambdaLedger`. The `setVestingLambda` function allows updating those bytes without any timelock. Since vestings do not pin a version, they always execute the newest bytes on the next call. A mistaken or malicious update to `lambdaClaim` or `lambdaAddVestee` propagates to all vestings immediately.

A threshold of signatories can replace claim or transfer logic instantly. This enables fund seizure or global bricking of vestings without notice.

Recommendation

We recommend introducing a timelock for lambda changes and version keys such as `name@version`. Pin each vesting to a version and require a timelocked migration to advance.

Status

Acknowledged

Centralisation

The Mavryk Vesting project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlock's analysis, the Mavryk project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au